

Report for ADL project
Motorized Automatic Pitcher For Guitar

Bogni Francesco - Sandrini Leonardo

June 30, 2026

Contents

1	Introduction	2
2	Signal reception and processing	3
2.1	Sampling	3
2.2	Downsampling	4
2.3	The ping pong buffer	5
2.4	The FFT IP	6
2.5	Smart peak finder	6
2.6	Simulation	9
2.7	Chapter summary	9
3	Data display	10
3.1	OLED display	10
3.2	Code implementation	10
3.3	Displayed information	10
4	Motor control	12
4.1	Component overview	12
4.2	Control logic	13
4.3	A4988 driver	13
4.4	NEMA 17 stepper motor	15
5	Conclusion and future work	16
5.1	Design Data	16
5.2	Future work	16

Chapter 1

Introduction

The integration of **digital signal processing** (DSP) and **automated systems** can offer solutions to possible real-world challenges. One example of said use cases involves the world of music, where the tuning of stringed instruments can be a frequent and tedious task for musicians. While digital tuners can provide visual feedback, they still require manual adjustment by the performer. This project introduces an autonomous, hardware-driven solution: an automatic guitar tuning system implemented on a **Xilinx Zedboard Development Kit** using **VHDL**.

The audio received directly from the guitar is processed thanks to a **Fast Fourier Transform** (FFT) algorithm, which identifies the fundamental frequency of the string being played. Such frequency is then compared to the desired target frequency, and allows the system to determine the necessary adjustments to bring the string into tune.

By bypassing general-purpose microprocessor and implementing the core logic directly in the **Programmable Logic** (PL) of the Zedboard, the design shows a possible full ASIC integration for the time audio analysis and motor control.

Signal reception and processing

The sampling of the data is performed using the ADAU1761 chip of the Zed-Board. It is a coder/decoder that samples the voltage level coming from the line-in, and can also output it on the line-out and do all sorts of transformations. The process of sampling has been facilitated by the use of the interface developed by Mike Field [2].



In Figure 2.1a and Figure 2.1b you can see the high level overview of the interface. We won't cover it in detail here, but you can check out the github repository [2].

The sampling frequency set by the interface is $48kHz$.

2.2 Downsampling

We aim to perform the Fast Fourier Transform (FFT) on our samples to detect what is the frequency component with the maximum power, i.e. the fundamental frequency. But what resolution can we achieve? What is the range of frequencies we can measure?

When we fill the FFT buffer, we are capturing a specific “window” of time. If we collect N samples, and take a new sample f_s times per second, the total time it takes to fill that buffer is:

$$T = \frac{N}{f_s} \quad (2.1)$$

The FFT assumes that the chunk of data we captured is one repeating cycle. Therefore, the absolute lowest frequency the FFT can see (other than 0 Hz DC) is a wave that completes exactly one full cycle inside our time window T . Thus:

$$f_{\min} = \frac{1}{T} \quad (2.2)$$

Substituting Equation 2.1 into Equation 2.2, we get:

$$f_{\min} = \frac{f_s}{N} = \Delta f \quad (2.3)$$

Since every bin of the FFT is spaced between one another in multiples of Δf .

We derived the equation that tells us the resolution **and** the minimum frequency detectable, but what about the maximum? The Nyquist theorem tells us that the maximum frequency we can detect through sampling is:

$$f_s > 2 \cdot f_{\max} \quad (2.4)$$

So with N fixed, to get a finer resolution (smaller Δf), we want to **lower** f_s , but not too much; otherwise, we will not be able to detect some needed frequencies according to Equation 2.4. *(The role of N will be discussed in Section 2.4)*

Here is where **downsampling** comes into play: we can decide how many samples to keep to achieve a certain resolution (instead of changing the sample rate, which would be inconvenient). But instead of discarding 9 samples out of 10, we accumulate those 10 values into a register and use that as our downsampled value.

This method also helps prevent spikes given by physical events (like picking a string) by effectively computing an *unscaled average*. (Since we are looking for the biggest value in the FFT, we do not need to scale it back down).

Moreover, it helps with another problem, **aliasing**: the Nyquist theorem (Equation 2.4) also tells us that if the guitar produces some high frequency noises (which it does), the low f_s will produce some low-frequency noise into the processed data. If you just dropped 9 out of 10 samples, all the frequencies multiple of f_s would alias to 0 Hz (DC), giving power to the incorrect bin. A visual explanation can be found at dsprelated.com[1].

The accumulation helps because it acts as a **low pass filter**: In signal processing, summing N subsequent samples is the same as **convolving** your audio with a “rectangular window” [3], which in the frequency domain has a frequency response given by the *sinc* function:

$$H(f) = \frac{\sin(\pi f M T_s)}{\sin(\pi f T_s)} \quad (2.5)$$

Where M is the downsample factor (10), and T_s is the sample period.

This solves completely the “alias to 0” problem, because the Equation 2.5 has zeroes at multiples of f_s . Moreover, the low-frequencies pass through with maximum gain ($M = 10$), while the higher frequencies are attenuated. For example, at our Nyquist limit:

$$H(2400) = 6.39 \quad (2.6)$$

Which is $\approx 63\%$ of maximum low-frequency amplitude.

2.3 The ping pong buffer

To prevent data loss while the FFT component is elaborating the data, we set up two buffers of 8192 entries using BRAM, and when one is full we feed

it to the FFT and start pushing new samples into the other buffer.

Since the effective f_s after downsampling is $4.8kHz$, then the time it takes to full one buffer is:

$$T = \frac{8192}{f_s} \approx 1.7s \quad (2.7)$$

Most of the time taken by the whole pipeline is waiting on the data to be sampled.

2.4 The FFT IP

To perform the FFT we use the Xilinx component **Fast Fourier Transform** 9.1. Reported here are the most important settings:

- Size (N) at 8192 bins
- Pipelined, streaming I/O

With these settings it uses 5185 LUTs, 34.5 BRAMs and 48 DSPs, and is able to do a full transformation in $247.15\mu s$. The output of the FFT are the frequencies real and complex components values spread across 8192 bins. N was chosen to be 8192 because it is a good compromise between logic area and **high** frequency resolution.

The Pipelined version was chosen over the Radix burst version only because it is much easier to control. We still are collecting the whole frame and then sending it to the FFT rather than sending a new sample whenever it comes, so switching to a Radix burst version can be done with not much struggle. Since we have area to spare we can't keep it that way. Quoting the most famous sentence in engineering: *if it works, don't touch it!*

2.5 Smart peak finder

To find the fundamental frequency we want to find the most powerful component. First problem is that the resulting 8192 bins, having the sample rate at 4800 Hz, are:

- Bins 0 to 4095 represent the true positive frequencies from 0 Hz to 2,400 Hz (Nyquist Limit).

- Bins 4096 to 8191 represent the negative frequencies wrapping backwards from 4,800 Hz down to 2,400 Hz.

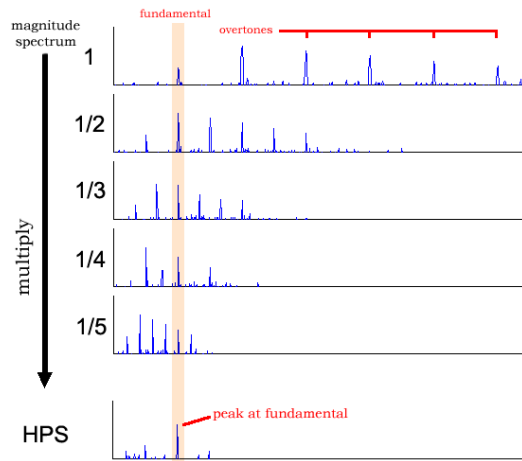
Because the spectrum is mirrored, the peak energy is split equally in half. If one plays a 440 Hz tone, the FFT produces a massive spike at Bin 751 (+440 Hz), but it produces an identical massive spike at Bin 7441 (which is $4,800 \text{ Hz} - 440 \text{ Hz} = 4360 \text{ Hz}$) That's why we only search for the bins up to 4096.

But here comes another problem: when playing the notes on a guitar, you don't only get the fundamental frequency, you also get multiples of it, the **harmonics**, which sometimes can have a higher power than the fundamental, breaking the calculation. To get around this we implemented an algorithm called **Harmonic Product Spectrum (HPS)**:

We hold 3 BRAMs, that will contain all the *real* samples of f_s , $2f_s$, $3f_s$ respectively of sizes 4096, 2048, 1366. When the data comes out of the FFT block, it puts all the bins in the first BRAM, one in two bins in the second and one in three for the third BRAM. As a result, the harmonics will line up at the same index, so we can multiply them together and finally

check for the **highest power**. A visual example of this can be seen in Figure 2.2

Figure 2.2: Example of HPS result. *Credits: sv.mazurka.org.uk*



There are different problems with this approach:

1. The full power calculation of the value of the bin takes 64 bits, and multiplying three of them together would result in a 128 bits value, taking way too many DSP slices over the ZedBoard and risking timing violations.
2. The operations of calculating the power and the HPS cannot be done in the same clock cycle: multiplication takes a long time.

For the **first** problem we first calculate the power with the standard formula, with the difference of avoiding the square root, since we only care about the maximum, there's no need to square it back down:

$$P = Re^2 + Im^2 \quad (2.8)$$

And from this 64 bits value we extract 16 bits, those between 48^{th} and 32^{nd} , which are the ones that carry the most useful information for us. If the full power value is bigger than 2^{48} we set the 16 bits to 1s. If instead the full power value is smaller than 2^{32} then we set the 16-bit value to 1 to avoid breaking the HPS multiplication.

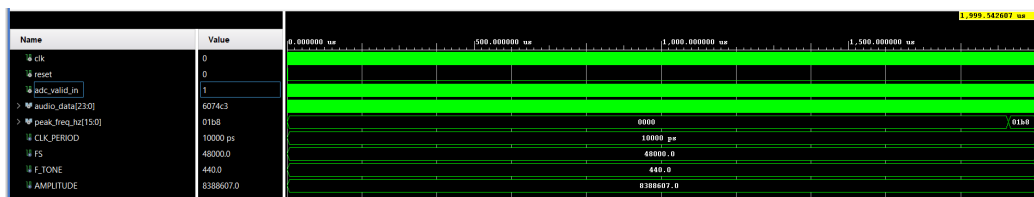
For the **second** problem we set up a “conveyor belt”: a 4 stage pipeline for the **write**:

1. Split real part and imaginary part.
2. Square each.
3. Add them together.
4. Extract the 16 bit value.

and a 3-stage pipeline for the **HPS**:

1. Multiply value in BRAM1 with value in BRAM2.
2. Multiply last result with value in BRAM3.
3. Compare and save the peak.

Figure 2.3: Screenshot of simulation results of `tb_fft_top`



2.6 Simulation

Reported in Figure 2.3 there's the result of the simulation of the `fft_top` component, which wires together all the parts in this chapter. Simulated is a sine wave at 440 hz, and fed to the fft. After filling the buffer the result `0x01b8` (In decimal 440) pops up as `peak_freq_hz`. More testbenches not reported here are available on [github](#).

2.7 Chapter summary

This is the conclusion to the part of the signal processing, that went from the very first sampling part to finding the fundamental frequency. It did so by downsampling the data to achieve a better resolution, setting up correct buffering to never do useless stalling, performing the FFT, and finding out the fundamental frequency overcoming many different problems. Now that we found that, we send the data to the OLED and the Stepper motor.

Chapter 3

Data display

3.1 OLED display

For quick access to information and data during testing, we used the **OLED display** integrated in the ZedBoard. This display has a resolution of 128x32 pixels and is controlled through I2C communication.

3.2 Code implementation

For the actual implementation of the OLED display, we used the following repository: [4] from Michael Mattioli, which provides a simple and effective way to interface with the display implemented in VHDL and compatible with the ZedBoard.

Many of the functions provided by the repository were kept unchanged, while we focused on **implementing new ones** to display the information we needed on the screen. The OLED display is divided into 4 lines, each with a maximum of 16 characters, and ASCII encoding is used to represent the characters.

3.3 Displayed information

Our audio processing module provides the **current frequency** of the audio signal. This data is encoded in **Binary-Coded Decimal** (BCD) format,

where each digit is written as 4-bits, which guarantee the necessary range from 0 to 9. The BCD format was chosen because it moves the operational weight of computing each digit from the OLED writer code, to other modules already tasked with computations. This allows a quick translation of the input data into an ascii encoded buffer, which will then be used to print the current frequency.

Another information displayed is **the note** corresponding to the current frequency. To achieve this we utilize the same data in BCD format and a **pre-compiled Look-Up Table** (LUT), containing the various notes in a specific frequency range. We chose the arbitrary value of 1000 Hz as the upper limit, which is more than enough to represent any relevant note detected by our system.

The last information displayed on the OLED display is a simple **incremental counter**, used for debugging purposes, like ensuring the right refresh rate on the display. This counter is also passed to the OLED module as a BCD encoded `std_logic_vector`.

Chapter 4

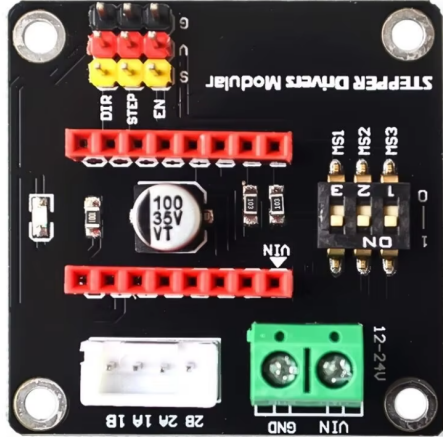
Motor control

4.1 Component overview

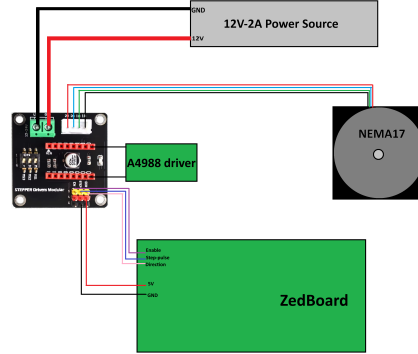
To interact with the physical world, our system utilizes a stepper motor and a specialized adapter to perform the tuning on the guitar.

- We utilize the **NEMA 17 stepper motor**, commonly used in 3D printers and similar electronic projects, due to its high torque and precision.
- The stepper motor is controlled by a **A4988 driver** [5], which allows for microstepping and a simplified use of the control signals.
- To support the driver a specialized **expansion board**, shown in Figure 4.1a was used, which simplified the control connections and power supply.
- A **custom 3D-printed adapter** was designed by taking inspiration from existing designs, and is used to connect the stepper motor to the guitar's tuning pegs.

A configuration of the implemented motor control system can be seen in Figure 4.1b.



(a) Driver expansion board



(b) System overview

4.2 Control logic

The stepper motor’s control logic is driven by the **100 MHz clock** of the ZedBoard, which is then divided internally inside the dedicated module. Extensive testing was performed to find the optimal frequency at which we could operate the motor to unleash enough **torque**, and for this goal we implemented a switch between 2 different hard-coded frequency divider values, allowing us to perform 2 test with each generation of the Bitstream. We settled on generating a step-pulse every 2.000.000 clock cycles, which corresponds to a final frequency of **25 Hz**.

Other than the clock divider, the control logic also produces the **direction** and **enable** signals, which decide the direction of rotation and whether the motor is active or not. These signals are generated as a result of the comparison between the target frequency and the current one, received from the audio processing module.

4.3 A4988 driver

The A4988 driver is a **microstepping driver**, designed to control bipolar stepper motors. It’s commonly used in 3D printers and similar projects, often paired with the NEMA 17. The driver is controlled by the 3 signals

`step`, `direction` and `enable`, generated by the control logic module, and is powered by a 5V power supply.

It allows to drive the stepper motor in smaller steps than its full step size, resulting in smoother and more precise movements, thanks to the **microstepping** feature. With the help of 3 switches on the driver's expansion board, the A4988 can be configured to divide each full step into **1, 2, 4, 8 or 16 microsteps**, trading speed for precision. To better utilize this functionality, the frequency of the step-pulse is divided by the chosen microstepping factor, and must be increased accordingly to maintain the same Rotations Per Minute (RPM) as the full step mode.

The relationship between the step-pulse frequency and the RPM of the motor is given by Equation 4.1

$$RPM = \frac{f_{step} \cdot 60}{steps_per_rev \cdot microstepping_factor} \quad (4.1)$$

Another advantage of the microstepping is that with the same RPM, the motor will receive more frequent pulses than in full step mode. This keeps it energized for a longer total amount of time, which results in a **higher torque** by avoiding long periods where no current is passing through the coils.

The A4988 driver also has a built-in current limiting feature, which with the use of a **potentiometer screw** mounted on the chip allows to set the maximum current that can flow through the motor coils, preventing damage and overheating. A thermal shutdown feature is also present, which disables the passing of current if the temperature exceeds a certain threshold, usually caused by a high current draw or insufficient cooling. Because of this features, physical tuning of the system had also to be performed to find the optimal current limit for the motor.

We first measured with a multimeter the resistance put up by the driver, while tuning the potentiometer, and then we calculated the current limit using the manufacturer's [5] Equation 4.2.

$$I_{max} = \frac{V_{ref}}{8 \cdot R_{sense}} \quad (4.2)$$

Wanting to achieve as much torque as possible, and consequently an higher current limit, we set the potentiometer to obtain a V_{ref} of 1.07 V. Considering the fixed R_{sense} of 0.1 Ω due to the internal resistors of the A4988, we calculated a current limit of 1.34 A.

4.4 NEMA 17 stepper motor

The **NEMA 17** is a bipolar stepper motor. In unipolar motors the current is drawn from a common connection, and sent to different sides of the coils by switching between the ground connections. In bipolar ones, the current is sent through the same path inside the coil, and the **polarity** must be reversed for each step to happen. This process requires a more complex circuit, complete of an H-Bridge, which is implemented inside the A4988 driver.

Bipolar stepper motors utilize the **totality** of the copper coil, which results in a **much higher torque** than unipolar motors, but can cause an increase in heat production. During the first phases of the project we tested the unipolar 28BYJ-48 stepper motor, a common and cheap solution for electronic's projects, but we quickly realized that the torque provided was insufficient to perform the tuning of the guitar.

The NEMA 17 has a full-step size of 1.8 degrees, taking 200 steps to complete a full rotation, with a step accuracy of ± 0.09 degrees. It has a rated Phase Inductance of 2.5 mH, a Phase Resistance of 1.5 Ω and a Phase Current of 1.5 A, which is the maximum current that can flow through the coils without causing damage or overheating.

The recommended operating voltage is 12 V, but an argument for the use of a 24 V supply could be made, since an higher voltage would guarantee a faster current rise time, and consequently an higher torque. However, this would increase the risk of overheating and would require a more complex current limiting circuit, like for example the DRV8825 driver.

Chapter 5

Conclusion and future work

5.1 Design Data

As a conclusion we report here the relevant data of the whole design: It takes in total 6308 LUTs, 9946 Flip Flops, 54 BRAMs and 60 DSPs. It has a 0.369 ns of Worst Negative Slack and an estimated total on-chip power of 0.511 W. The motor has an estimated power utilization of 16.08 W while in use.

5.2 Future work

As a next step one could modify the design to lower the FFT resolution to use less area but most importantly less time to fill the buffer, to give the user the experience of having a continuous signal analysis rather than a discrete one. One could also try to make the FFT streaming rather than waiting for the buffer, or Radix burst to use less area. It is also possible to lower the sampling frequency with the benefits and drawbacks described in Chapter 2.

Once the structure is built, building new feature on it is much easier. You could add more motors to tune every string together (although it comes with some changes to the signal processing part), or add a more interactive UI; limit's the sky.

In conclusion one can experiment tuning this parameters to achieve the goal they want, or build new features on it but this design stays moreover the same. This report can serve as a guideline to whomever wants to improve it.

Bibliography

- [1] dsprelated.com. <https://www.dsprelated.com/foundations/sampling.php>.
- [2] Mike Field. zedboard audio interface. https://github.com/ems-kl/zedboard_audio, 2015.
- [3] Richard G. Lyons. *Understanding Digital Signal Processing*, section 11.5.
- [4] Michael Mattioli. Oled repository. <https://github.com/mmattioli/ZedBoard-OLED.git>.
- [5] Allegro MicroSystems. A4988 datasheet. <https://www.allegromicro.com/~media/files/datasheets/a4988-datasheet.pdf>, 2022.